```
Output:
string(11) "👨"
string(22) "f09f91a8e2808df09fa6b2"
```

Again, there's a chance your system will produce different output when rendering a multi-codepoint character. Let's break down the output of `bin2hex()` and see how it maps to UTF-8.

Table 8-7. *UTF-8 code units for "Bald Man"*

UTF-8 code unit	Unicode codepoint	Character
F0 9F 91 A9	U+1F468	👨 (Man)
E2 80 8D	U+200D	ZERO_WIDTH_JOINER
F0 9F A6 B2	U+1F9B2	🦲 (Bald)

PHP Unicode escapes will always generate UTF-8 code units, but it's still just a byte stream, and you still need a UTF-8-compliant browser/editor/console to view the bald man. (If you have any particular interest in the recent emoji additions to Unicode 15.1, start here: https://unicode.org/emoji/charts/.)

Comparison operators ==, <, and > (more detail in Chapter 4, "Logic and Control") will also work with string buffers, as will built-in functions that use comparison internally, such as the array-sorting function `sort()`:

```
$things = ['Zebra', 'Apple', 'Orange', 'banana'];
sort($things);
print_r($things);
```

```
Output:
Array
(
    [0] => Apple
    [1] => Orange
    [2] => Zebra
    [3] => banana
)
```